



TOWARDS A FOUNDATIONAL MODEL FOR PARTICLE-BASED METHODS

Scaling neural operators for large-scale plasma simulations and coupling to production codes

October 2026 | Nikola Mamić | JSC

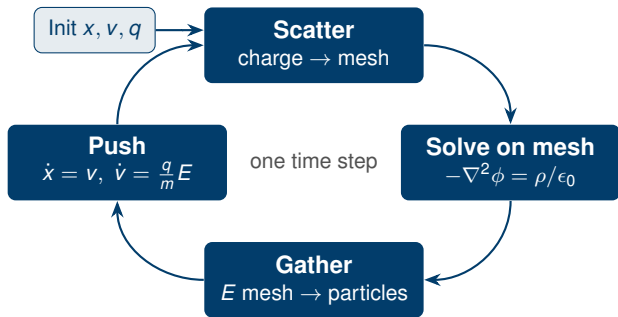


Introduction

TOWARDS A FOUNDATIONAL MODEL FOR PARTICLE-BASED METHODS

PARTICLE–FIELD SYSTEMS AND THE PIC LOOP

- **Particles** move by ODEs, **fields** obey PDEs
- Coupled in both directions: particles make the field, the field pushes the particles
- Plasmas (particle-in-cell, PIC), vortex flows, ...
- Electrostatic plasma:
 $-\nabla^2 \phi = \rho / \epsilon_0, \quad E = -\nabla \phi$



FROM GRIDS TO FOURIER MODES

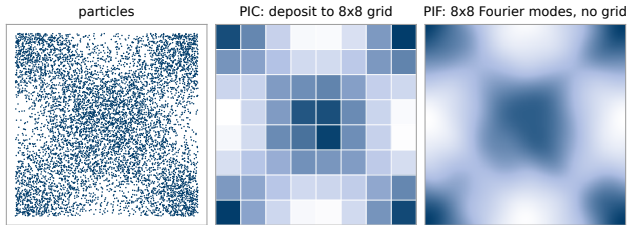
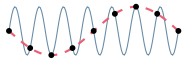


Illustration with synthetic particles, both with 64 degrees of freedom

PIC: the grid causes trouble



aliasing → grid
heating, energy drift



cells must resolve
the **Debye length**

PIF: particles talk to Fourier modes

- $\hat{\rho}_k = S_k \sum_j q_j e^{-ik \cdot x_j}$ - no real-space grid
- No grid aliasing, spectral accuracy
- With NUFFT: same cost order as PIC, higher constant
- Limits: global operation; periodic / free-space boundaries

WHY LEARN THE FIELD OPERATOR?

$\sim 10^{11}$
grid points / modes

$\sim 10^{12}$
particles

$\sim 10^5$
time steps

for one high-fidelity run (fusion, particle accelerators)

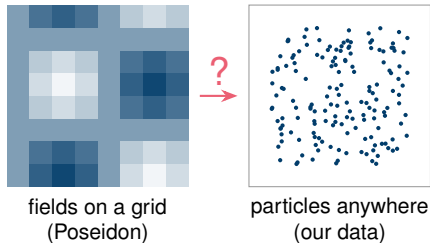
Hypotheses of the NEOPIC project

- PIC and PIF **do** scale well on today's supercomputers
- High-fidelity and many-query studies remain **too expensive**
- Implicit / larger time steps via autograd Jacobians
- Differentiable simulations
- Field operators that are expensive or not known exactly
- One model for many codes

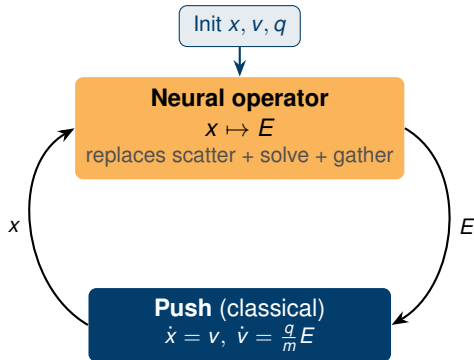
By operation count, one inference step costs more than one PIF step: the model applies the PIF-type transform $2L$ times per step for d_v channels. Whether that pays off over a whole simulation is open; **scaling and coupling** are the prerequisites to find out.

DEEP LEARNING SURROGATES

- **Foundation models**: pretrain once, fine-tune for new tasks
- **Function to function**: not tied to one grid or resolution"
- Precedent - **Poseidon** (Herde et al., NeurIPS 2024): pretrained on a **small** set of fluid PDEs (compressible Euler, incompressible Navier–Stokes), generalises to unseen PDEs
- **Differentiable** end to end: gradients for optimisation, inverse problems, implicit time stepping

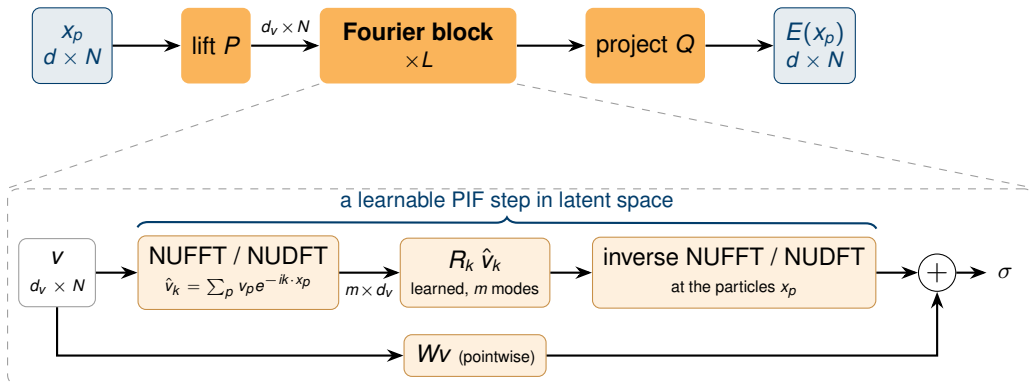


THE PROJECT'S APPROACH



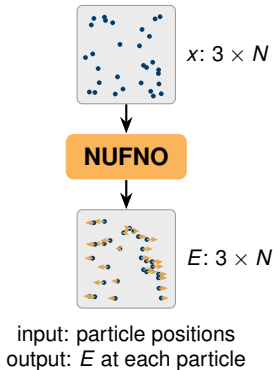
- Learn the whole map $P^T \mathcal{L}^{-1} P : x \mapsto E$
electrostatic: E depends on the positions only
- Particle push stays classical
- Training data from mini-apps (PIF/PIC)
- Input: particle positions $\rightarrow$ output: E at the particles

THE MODEL: NONUNIFORM FNO

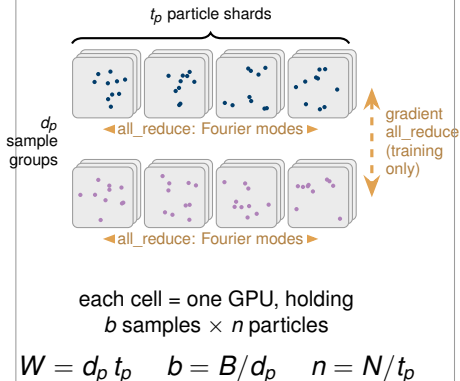


WHAT GOES IN, WHAT COMES OUT

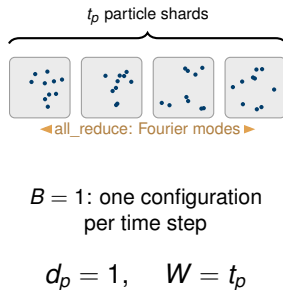
One sample



Training



Inference





The project and the contribution

TOWARDS A FOUNDATIONAL MODEL FOR PARTICLE-BASED METHODS

THE PROJECT

- NUFNO model
- Distributed training: data parallel (d_p) $\times$ particle decomposition (t_p)
- t_p (tensor parallel in the code) = particle decomposition
- picND: evaluation against a PIF reference
- Training data from the mini-apps

THE PROJECT

- NUFNO model
- Distributed training: data parallel (d_p) $\times$ particle decomposition (t_p)
- t_p (tensor parallel in the code) = particle decomposition
- picND: evaluation against a PIF reference
- Training data from the mini-apps

The contributions made:

Scaling inference out

from one GPU holding all particles to many GPUs

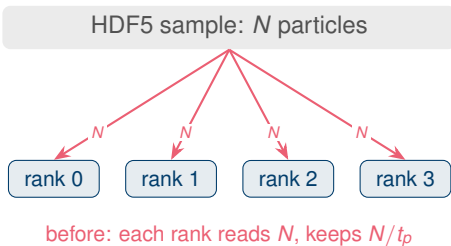
Leaner I/O

each rank reads only the particles it needs

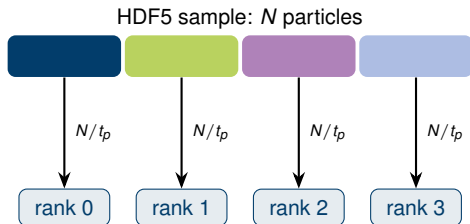
Into IPPL

run the model as a surrogate inside a simulation library

TRAINING I/O: SHARDED HDF5 READS



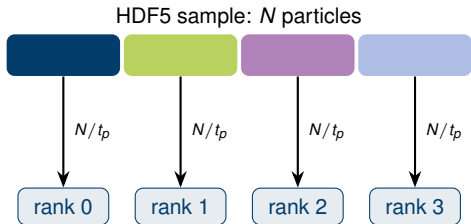
TRAINING I/O: SHARDED HDF5 READS



after: each rank reads only its shard

- Read volume per rank: $N \rightarrow N/t_p$

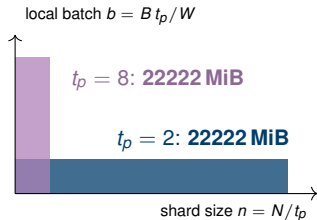
TRAINING I/O: SHARDED HDF5 READS



- Read volume per rank: $N \rightarrow N/t_p$
- Peak GPU memory **unchanged**: set downstream by activations / Fourier layers

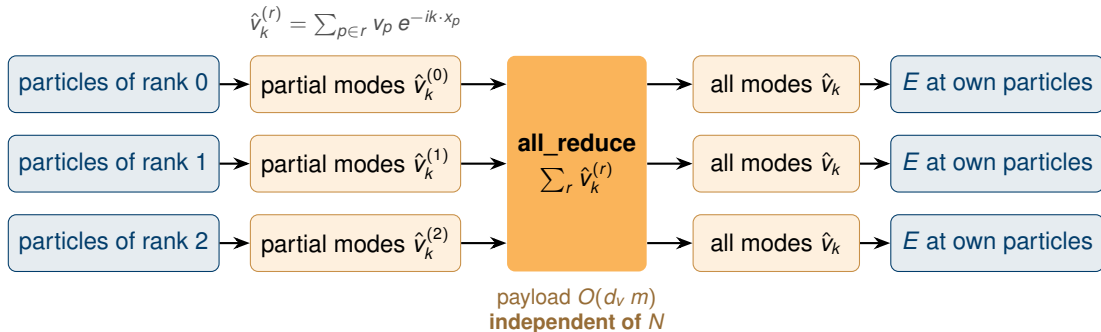
Why a larger t_p did not lower peak memory

($W = 8$ GPUs fixed)



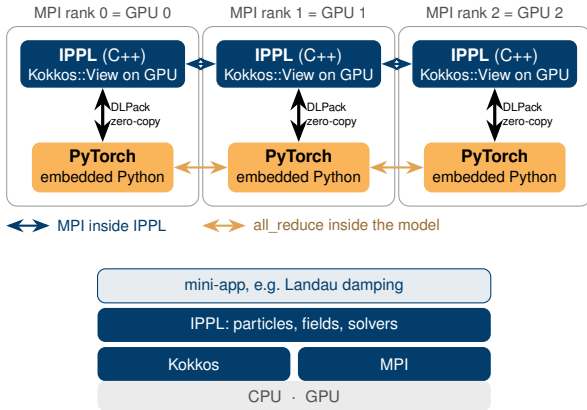
$$b \cdot n = \frac{BN}{W} \text{ independent of } t_p$$

PARALLEL INFERENCE



- Particle decomposition in `picND`: each GPU holds N/W particles; in inference $W = t_p$ (no data parallelism)
- Integrated quantities (e.g. field energy) also via `all_reduce`

COUPLING TO IPPL



- IPPL: C++ particle–mesh library, performance portable (Kokkos + MPI); underlies the production accelerator code OPAL-X
- Time step unchanged: only the **field evaluation** is swapped for the model
- Opt-in, header-only (pybind11 + DLPack), **no changes to IPPL**
- Same MPI rank hosts IPPL and the model: **rank i talks to rank i**
- Data stays on the GPU; Python writes E **directly into IPPL's memory**

SCALING RESULTS

NUFFT model, run **through the IPPL interop layer**: 400 time steps, 1–512 GPUs, up to 1.6 B particles

FIGURE: Strong + weak scaling

IPPL-interop deck p.13

[figures/ippl_inference_scaling.pdf](#)|png

FIGURE: Scaling efficiency

IPPL-interop deck p.14

[figures/ippl_scaling_efficiency.pdf](#)|png

efficiency: strong $E_s = P_0 T(P_0) / (P T(P))$, weak $E_w = T_{\text{base}} / T(P)$



Deep dive

TOWARDS A FOUNDATIONAL MODEL FOR PARTICLE-BASED METHODS

TWO WAYS TO SPLIT THE PARTICLES

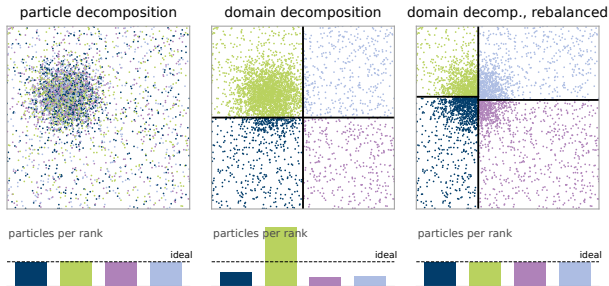


Illustration with synthetic particles (beam on a background), 4 ranks

Particle decomposition (The project's model, picND)

each rank: random subset over the **whole** domain; always balanced; Fourier modes duplicated

Domain decomposition (IPPL)

each rank: **one subdomain**; balance needs re-partitioning
→ subdomain boundaries move

Under domain decomposition, clustered distributions also unbalance the **model's** work per rank.

THE BUG: LOCAL VS. GLOBAL BOUNDS

```
# valid(): global bounds
lo = all_reduce(x.min(), MIN)
hi = all_reduce(x.max(), MAX)

# __call__(): local bounds
lo, hi = x.min(), x.max()
```

schematic, not the actual code

- Particle decomp.: local $\approx$ global bounds $\rightarrow$ **harmless**, invisible in picND
- Domain decomp.: each rank stretches its subdomain to $[0, 2\pi)$ $\rightarrow$ ranks sum modes in **incompatible coordinates**
- **Fix**: pass global bounds (one `all_reduce`, or the known domain size)

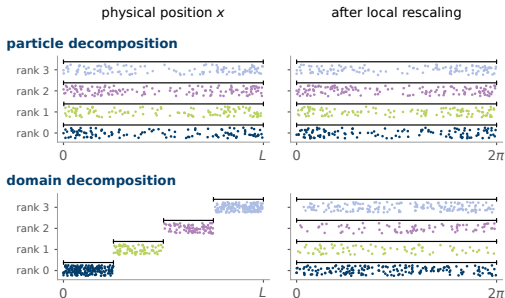


Illustration: synthetic 1D density $1 + 0.6 \cos x$, 4 ranks, $|k| \leq 4$

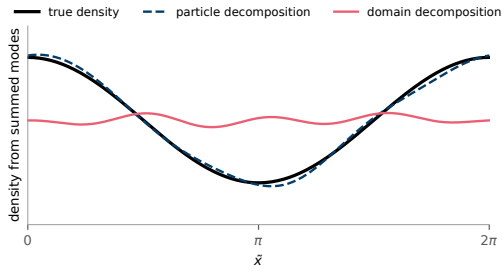
THE BUG: LOCAL VS. GLOBAL BOUNDS

```
# valid(): global bounds
lo = all_reduce(x.min(), MIN)
hi = all_reduce(x.max(), MAX)

# __call__(): local bounds
lo, hi = x.min(), x.max()
```

schematic, not the actual code

- Particle decomp.: local $\approx$ global bounds $\rightarrow$ **harmless**, invisible in picND
- Domain decomp.: each rank stretches its subdomain to $[0, 2\pi)$ $\rightarrow$ ranks sum modes in **incompatible coordinates**
- **Fix**: pass global bounds (one `all_reduce`, or the known domain size)



Density rebuilt from the all-reduced modes

Illustration: synthetic 1D density $1 + 0.6 \cos x$, 4 ranks, $|k| \leq 4$

The Fourier sum does not care which rank holds which particle:
with consistent coordinates the model works under **any**
decomposition, including IPPL's.

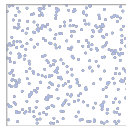
THE RISING NOISE FLOOR

```
# before: same seed on every rank  
cp.random.seed(152)
```

```
# fix: one seed per rank  
cp.random.seed(152 + 100*rank)
```

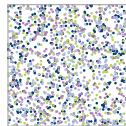
- Every GPU draws its N/W particles from the **same seed** → very similar particles on all ranks
- Effectively only $\approx N/W$ independent particles → noise floor **grows with the GPU count**
- One seed per rank → the noise floor no longer depends on W

seed(152) on every rank



400 distinct particles

seed(152 + 100*rank)



1600 distinct particles

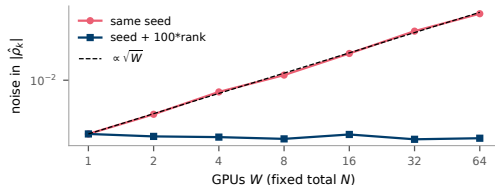
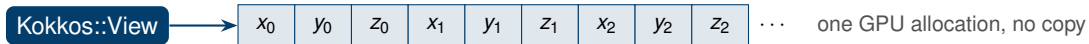
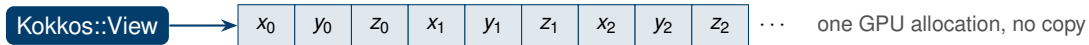


Illustration: numpy, uniform particles, fixed $N = 2^{16}$, modes $k = 1..8$, 20 trials

ZERO-COPY VIA DLPACK



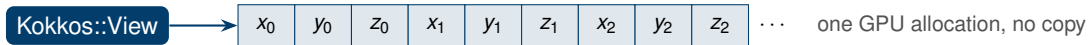
ZERO-COPY VIA DLPACK



DLTensor{ data, device=CUDA,
dtype=float64, shape=(N,3),
strides=(3,1) } + deleter

Zero-copy only if standard layout, no padding:
`sizeof(Vector<T,D>) == D*sizeof(T)`
otherwise: **repack on the device** (still no host copy)

ZERO-COPY VIA DLPACK



DLTensor{ data, device=CUDA,
dtype=float64, shape=(N,3),
strides=(3,1) } + deleter

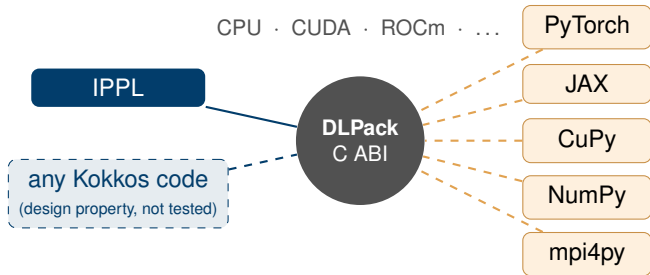
Zero-copy only if standard layout, no padding:
`sizeof(Vector<T,D>) == D*sizeof(T)`
otherwise: **repack on the device** (still no host copy)

Lifetime

the capsule keeps the Kokkos View alive until PyTorch releases it

ZERO-COPY VIA DLPACK

Why it matters beyond this project



- Open standard: NumPy, CuPy, PyTorch, TensorFlow, JAX, MXNet, TVM, Paddle, mpi4py
- Cross-hardware, plain C ABI → any language
- Our export side is generic over Kokkos Views; model side not tied to PyTorch



Status

TOWARDS A FOUNDATIONAL MODEL FOR PARTICLE-BASED METHODS

STATUS AND OUTLOOK

Works

- Sharded HDF5 reads in training
- Sharded inference in `picND`
- One RNG seed per rank
- Global-bounds fix
- IPPL $\leftrightarrow$ PyTorch coupling, end to end
- Inference scaling through IPPL: up to 512 GPUs, 1.6 B particles

In progress

- ADIOS2 / openPMD streaming

Next

- Per-step cost comparison vs. IPPL's PIF at scale
- Mixed precision
- Load balancing for clustered distributions
- Testing the time-step hypothesis

Open question: does the learned operator pay off over a full simulation compared to PIF?

ACKNOWLEDGEMENTS

- **Dr. Sriramkrishnan Muralikrishnan** (JSC) – supervisor
- **Chelsea John** – the foundation model is her PhD thesis; my work contributes to it
- NEOPIC is funded by **Helmholtz AI**
INF funding code ZT-I-PF-5-242

Thank you!

Questions?